

Système de gestion de facturation et paiements

Étape 6 - Conception de la base de données

Élément	Description
Type de document	Conception technique de la base de données
Version	V1 - Première version du système
Objectif	Définir les entités, tables, relations, contraintes, règles de calcul et schéma SQL de départ
Statut	Base de travail pour l'architecture technique et le développement

1. Objectif de cette étape

La conception de la base de données transforme les exigences, les user stories et les cas d'utilisation en une structure technique exploitable par les développeurs. Elle définit les tables, les relations, les clés primaires, les clés étrangères, les contraintes et les règles de calcul.

La base de données doit supporter les modules de la V1 : utilisateurs, rôles, clients, articles ou services, factures, lignes de facture, paiements, relevés de compte, transactions extraites, commissions, rapports et historique des actions.

2. Choix recommandé de base de données

2.1 Recommandation

PostgreSQL

PostgreSQL est recommandé pour cette application parce qu'il est fiable pour les données financières, robuste pour les contraintes relationnelles, adapté aux transactions et approprié pour conserver des historiques fiables. MySQL peut aussi fonctionner, mais PostgreSQL est préférable pour une application professionnelle de facturation.

3. Entités principales

- roles
- users
- permissions
- role_permissions
- clients
- items
- invoices
- invoice_items
- payments
- bank_statements
- bank_transactions
- commissions
- audit_logs
- settings

4. Vue générale des relations

```
roles 1 ---- * users
roles * ---- * permissions via role_permissions
clients 1 ---- * invoices
invoices 1 ---- * invoice_items
items 1 ---- * invoice_items
invoices 1 ---- * payments
bank_statements 1 ---- * bank_transactions
bank_transactions 0..1 ---- 0..1 invoices
invoices 1 ---- * commissions
users 1 ---- * audit_logs
```

5. Tables du module Utilisateurs et sécurité

5.1 Table roles

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique du rôle
name	VARCHAR(50)	Nom du rôle : admin, employee
description	TEXT	Description du rôle
created_at	TIMESTAMP	Date de création
updated_at	TIMESTAMP	Date de modification

5.2 Table users

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique de l'utilisateur
role_id	FK	Référence vers roles.id
full_name	VARCHAR(150)	Nom complet
email	VARCHAR(150)	Email de connexion, unique si utilisé
username	VARCHAR(100)	Identifiant de connexion unique
password_hash	VARCHAR(255)	Mot de passe chiffré
status	VARCHAR(30)	actif, désactivé
last_login_at	TIMESTAMP	Dernière connexion
created_at / updated_at	TIMESTAMP	Dates de création et modification

Règles : le mot de passe ne doit jamais être stocké en clair ; chaque utilisateur doit avoir un rôle ; un utilisateur désactivé ne peut pas se connecter.

5.3 Tables permissions et role_permissions

Ces tables permettent une gestion plus fine des droits. Elles seront utiles pour distinguer ce qu'un administrateur ou un employé peut réellement faire.

```
permissions: id, code, description
role_permissions: id, role_id, permission_id
```

Exemples de permissions:
 client.create, client.update, client.delete, client.archive,
 invoice.create, invoice.cancel, payment.create, report.view, user.manage

6. Module Clients

6.1 Table clients

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique du client
client_code	VARCHAR(50)	Code client unique, optionnel
full_name	VARCHAR(150)	Nom du client, obligatoire
phone	VARCHAR(50)	Numéro de téléphone
email	VARCHAR(150)	Email
address	TEXT	Adresse
client_type	VARCHAR(30)	particulier, entreprise
status	VARCHAR(30)	actif, archivé
notes	TEXT	Notes internes
created_by	FK	Utilisateur ayant créé le client

Champ	Type	Description
created_at / updated_at	TIMESTAMP	Dates de création et modification
archived_at	TIMESTAMP	Date d'archivage

Règles métier : le nom est obligatoire ; un client est actif par défaut ; un client avec facture ou paiement ne doit pas être supprimé définitivement ; un client archivé reste consultable mais ne doit pas apparaître par défaut lors de la création d'une facture.

7. Module Articles ou services

7.1 Table items

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
name	VARCHAR(150)	Nom de l'article ou service
description	TEXT	Description
item_type	VARCHAR(30)	article, service
default_price	DECIMAL(12,2)	Prix par défaut
status	VARCHAR(30)	actif, désactivé
created_by	FK	Utilisateur créateur
created_at / updated_at	TIMESTAMP	Dates de création et modification
disabled_at	TIMESTAMP	Date de désactivation

Règles métier : le nom est obligatoire ; le prix ne peut pas être négatif ; un article ou service désactivé ne doit pas être proposé par défaut ; un article déjà utilisé dans une facture ne doit pas être supprimé définitivement.

8. Module Factures

8.1 Table invoices

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
invoice_number	VARCHAR(50)	Numéro unique de facture
client_id	FK	Référence vers clients.id
status	VARCHAR(50)	brouillon, envoyée, payée, partiellement_payée, non_payée, annulée
issue_date	DATE	Date de création officielle
due_date	DATE	Date limite de paiement, optionnel
subtotal_amount	DECIMAL(12,2)	Total avant réductions/taxes, si applicable
total_amount	DECIMAL(12,2)	Montant total
paid_amount	DECIMAL(12,2)	Montant payé
balance_due	DECIMAL(12,2)	Solde restant
notes	TEXT	Notes
pdf_url	VARCHAR(255)	Chemin ou lien du PDF généré
created_by	FK	Utilisateur créateur
cancelled_at / cancelled_by	TIMESTAMP / FK	Informations d'annulation
cancellation_reason	TEXT	Raison d'annulation

Règles métier : chaque facture générée doit avoir un numéro unique ; elle doit être liée à un client ; elle doit contenir au moins une ligne avant génération officielle ; une facture brouillon peut être modifiée ; une facture générée ne doit pas être supprimée ; une facture annulée reste visible dans l'historique.

8.2 Table invoice_items

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
invoice_id	FK	Référence vers invoices.id
item_id	FK nullable	Référence vers items.id
item_name	VARCHAR(150)	Nom copié au moment de la facture
description	TEXT	Description copiée ou personnalisée
quantity	DECIMAL(10,2)	Quantité
unit_price	DECIMAL(12,2)	Prix unitaire utilisé
line_total	DECIMAL(12,2)	Total de la ligne
payment_status	VARCHAR(50)	payé, partiellement_payé, non_payé
created_at / updated_at	TIMESTAMP	Dates de création et modification

Règle clé : les informations de l'article ou service doivent être copiées dans la ligne de facture au moment de la création. Ainsi, si le prix d'un service change plus tard, les anciennes factures ne changent pas.

9. Module Paiements

9.1 Table payments

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
invoice_id	FK	Référence vers invoices.id
client_id	FK	Référence vers clients.id
amount	DECIMAL(12,2)	Montant payé
payment_date	DATE	Date du paiement
payment_method	VARCHAR(50)	cash, virement_bancaire, mobile_money, carte_bancaire, chèque, autre
reference	VARCHAR(150)	Référence du paiement
notes	TEXT	Notes
created_by	FK	Utilisateur ayant enregistré
created_at / updated_at	TIMESTAMP	Dates d'enregistrement et modification

Règles métier : un paiement doit être lié à une facture ; son montant doit être supérieur à zéro ; il ne doit pas dépasser le solde restant ; après chaque paiement, le solde et le statut de la facture sont mis à jour.

10. Module Relevés de compte

10.1 Table bank_statements

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
file_name	VARCHAR(255)	Nom du fichier
file_url	VARCHAR(255)	Emplacement du fichier

Champ	Type	Description
source_type	VARCHAR(50)	upload, scan
status	VARCHAR(50)	importé, traité, erreur
imported_by	FK	Utilisateur ayant importé
imported_at / processed_at	TIMESTAMP	Dates d'import et de traitement
notes	TEXT	Notes

10.2 Table bank_transactions

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
bank_statement_id	FK	Référence vers bank_statements.id
extracted_client_name	VARCHAR(150)	Nom extrait du relevé
matched_client_id	FK nullable	Client correspondant, si trouvé
transaction_date	DATE	Date de transaction
amount	DECIMAL(12,2)	Montant de la transaction
reference	VARCHAR(150)	Référence bancaire
raw_text	TEXT	Texte brut extrait
status	VARCHAR(50)	extrait, corrigé, validé, utilisé, rejeté
correction_notes	TEXT	Notes de correction
validated_by / validated_at	FK / TIMESTAMP	Validation humaine
created_invoice_id	FK nullable	Facture créée depuis cette transaction

Règles métier : une transaction extraite ne crée pas automatiquement une facture ; la validation humaine est obligatoire ; une transaction déjà utilisée ne doit pas créer deux factures par erreur.

11. Module Commissions

11.1 Table commissions

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
invoice_id	FK	Référence vers invoices.id
intermediary_name	VARCHAR(150)	Nom de l'intermédiaire
commission_type	VARCHAR(50)	fixe, pourcentage
commission_value	DECIMAL(12,2)	Valeur saisie
calculated_amount	DECIMAL(12,2)	Montant calculé
paid_amount	DECIMAL(12,2)	Montant déjà payé
balance_due	DECIMAL(12,2)	Reste à payer
payment_status	VARCHAR(50)	non_payée, partiellement_payée, payée
notes	TEXT	Notes
created_by	FK	Utilisateur créateur
created_at / updated_at	TIMESTAMP	Dates de création et modification

Règles métier : une commission doit être liée à une facture ; elle peut être fixe ou en pourcentage ; si elle est en pourcentage, son montant est calculé à partir du total de la facture ; son historique doit être conservé.

12. Historique et audit

12.1 Table audit_logs

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
user_id	FK	Utilisateur ayant fait l'action
action	VARCHAR(100)	Type d'action
entity_type	VARCHAR(100)	Type d'objet concerné
entity_id	UUID / BIGINT	Identifiant de l'objet concerné
old_values	JSONB	Anciennes valeurs
new_values	JSONB	Nouvelles valeurs
ip_address	VARCHAR(100)	Adresse IP
created_at	TIMESTAMP	Date de l'action

Actions à enregistrer:
client_created, client_updated, client_archived, client_deleted,
invoice_created, invoice_updated, invoice_cancelled,
payment_created, commission_created,
bank_statement_imported, bank_transaction_validated,
user_created, user_disabled

13. Table settings

Champ	Type	Description
id	UUID / BIGINT	Identifiant unique
key	VARCHAR	Clé du paramètre
value	TEXT	Valeur
description	TEXT	Description
updated_by	FK	Utilisateur ayant modifié
updated_at	TIMESTAMP	Date de modification

Exemples de paramètres:
invoice_prefix = FAC
invoice_next_number = 000001
default_currency = USD
company_name = Nom de l'entreprise
company_address = Adresse de l'entreprise

14. Diagramme relationnel textuel

roles
-> users

roles
-> role_permissions
-> permissions

users
-> clients.created_by
-> items.created_by
-> invoices.created_by
-> payments.created_by
-> commissions.created_by
-> audit_logs.user_id

clients
-> invoices
-> invoice_items
-> items

```
invoices
-> payments
-> commissions

bank_statements
-> bank_transactions
-> clients
-> invoices
```

15. Contraintes importantes

15.1 Clés primaires

Chaque table doit avoir une clé primaire id. La recommandation est d'utiliser UUID afin de réduire les risques de collision et de faciliter l'évolution future du système.

15.2 Clés étrangères

```
users.role_id -> roles.id
invoices.client_id -> clients.id
invoice_items.invoice_id -> invoices.id
invoice_items.item_id -> items.id
payments.invoice_id -> invoices.id
payments.client_id -> clients.id
commissions.invoice_id -> invoices.id
bank_transactions.bank_statement_id -> bank_statements.id
bank_transactions.matched_client_id -> clients.id
bank_transactions.created_invoice_id -> invoices.id
```

15.3 Unicité

```
users.email doit être unique si utilisé.
users.username doit être unique.
invoices.invoice_number doit être unique.
clients.client_code doit être unique si utilisé.
permissions.code doit être unique.
```

15.4 Contraintes de montants

```
items.default_price >= 0
invoice_items.quantity > 0
invoice_items.unit_price >= 0
invoice_items.line_total >= 0
invoices.total_amount >= 0
invoices.paid_amount >= 0
invoices.balance_due >= 0
payments.amount > 0
commissions.calculated_amount >= 0
```

16. Calculs importants

```
line_total = quantity x unit_price
invoice.total_amount = somme des invoice_items.line_total
invoice.paid_amount = somme des payments.amount liés à la facture
invoice.balance_due = invoice.total_amount - invoice.paid_amount
```

Détermination du statut:

Si status = annulée: conserver annulée

Sinon si paid_amount = 0: non_payée

Sinon si paid_amount < total_amount: partiellement_payée

Sinon si paid_amount = total_amount: payée

17. Index recommandés

```
users.email
users.username
clients.full_name
clients.phone
clients.status
items.name
items.status
invoices.invoice_number
invoices.client_id
```



```

invoices.status
invoices.issue_date
payments.invoice_id
payments.client_id
bank_transactions.status
bank_transactions.matched_client_id
commissions.invoice_id
audit_logs.entity_type
audit_logs.entity_id

```

18. Suppression, archivage et conservation

Objet	Règle recommandée
Client sans historique	Suppression possible
Client avec historique	Archivage obligatoire
Article jamais utilisé	Suppression possible
Article déjà utilisé	Désactivation obligatoire
Facture générée	Suppression définitive interdite ; annulation autorisée
Païement enregistré	Conservation obligatoire dans l'historique
Relevés et transactions	Conserver fichiers, extractions, corrections et validations

19. Schéma SQL simplifié

```

CREATE TABLE roles (
  id UUID PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  description TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP
);

CREATE TABLE users (
  id UUID PRIMARY KEY,
  role_id UUID NOT NULL REFERENCES roles(id),
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(150) UNIQUE,
  username VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'actif',
  last_login_at TIMESTAMP,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP
);

CREATE TABLE clients (
  id UUID PRIMARY KEY,
  client_code VARCHAR(50) UNIQUE,
  full_name VARCHAR(150) NOT NULL,
  phone VARCHAR(50),
  email VARCHAR(150),
  address TEXT,
  client_type VARCHAR(30) DEFAULT 'particulier',
  status VARCHAR(30) NOT NULL DEFAULT 'actif',
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP,
  archived_at TIMESTAMP
);

CREATE TABLE items (
  id UUID PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  description TEXT,
  item_type VARCHAR(30) NOT NULL,
  default_price DECIMAL(12,2) NOT NULL DEFAULT 0,
  status VARCHAR(30) NOT NULL DEFAULT 'actif',
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```

        updated_at TIMESTAMP,
        disabled_at TIMESTAMP,
        CHECK (default_price >= 0)
    );

CREATE TABLE invoices (
    id UUID PRIMARY KEY,
    invoice_number VARCHAR(50) UNIQUE,
    client_id UUID NOT NULL REFERENCES clients(id),
    status VARCHAR(50) NOT NULL DEFAULT 'brouillon',
    issue_date DATE,
    due_date DATE,
    subtotal_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
    total_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
    paid_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
    balance_due DECIMAL(12,2) NOT NULL DEFAULT 0,
    notes TEXT,
    pdf_url VARCHAR(255),
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    cancelled_at TIMESTAMP,
    cancelled_by UUID REFERENCES users(id),
    cancellation_reason TEXT,
    CHECK (total_amount >= 0),
    CHECK (paid_amount >= 0),
    CHECK (balance_due >= 0)
);

CREATE TABLE invoice_items (
    id UUID PRIMARY KEY,
    invoice_id UUID NOT NULL REFERENCES invoices(id),
    item_id UUID REFERENCES items(id),
    item_name VARCHAR(150) NOT NULL,
    description TEXT,
    quantity DECIMAL(10,2) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    line_total DECIMAL(12,2) NOT NULL,
    payment_status VARCHAR(50) NOT NULL DEFAULT 'non_payé',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    CHECK (quantity > 0),
    CHECK (unit_price >= 0),
    CHECK (line_total >= 0)
);

CREATE TABLE payments (
    id UUID PRIMARY KEY,
    invoice_id UUID NOT NULL REFERENCES invoices(id),
    client_id UUID NOT NULL REFERENCES clients(id),
    amount DECIMAL(12,2) NOT NULL,
    payment_date DATE NOT NULL,
    payment_method VARCHAR(50) NOT NULL,
    reference VARCHAR(150),
    notes TEXT,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    CHECK (amount > 0)
);

CREATE TABLE bank_statements (
    id UUID PRIMARY KEY,
    file_name VARCHAR(255) NOT NULL,
    file_url VARCHAR(255),
    source_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'importé',
    imported_by UUID REFERENCES users(id),
    imported_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_at TIMESTAMP,
    notes TEXT
);

CREATE TABLE bank_transactions (
    id UUID PRIMARY KEY,
    bank_statement_id UUID NOT NULL REFERENCES bank_statements(id),
    extracted_client_name VARCHAR(150),
    matched_client_id UUID REFERENCES clients(id),
    transaction_date DATE,
    amount DECIMAL(12,2) NOT NULL,

```

```

reference VARCHAR(150),
raw_text TEXT,
status VARCHAR(50) NOT NULL DEFAULT 'extrait',
correction_notes TEXT,
validated_by UUID REFERENCES users(id),
validated_at TIMESTAMP,
created_invoice_id UUID REFERENCES invoices(id),
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP,
CHECK (amount >= 0)
);

CREATE TABLE commissions (
  id UUID PRIMARY KEY,
  invoice_id UUID NOT NULL REFERENCES invoices(id),
  intermediary_name VARCHAR(150) NOT NULL,
  commission_type VARCHAR(50) NOT NULL,
  commission_value DECIMAL(12,2) NOT NULL,
  calculated_amount DECIMAL(12,2) NOT NULL,
  paid_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
  balance_due DECIMAL(12,2) NOT NULL DEFAULT 0,
  payment_status VARCHAR(50) NOT NULL DEFAULT 'non_payée',
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP,
  CHECK (commission_value >= 0),
  CHECK (calculated_amount >= 0),
  CHECK (paid_amount >= 0),
  CHECK (balance_due >= 0)
);

CREATE TABLE audit_logs (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  action VARCHAR(100) NOT NULL,
  entity_type VARCHAR(100) NOT NULL,
  entity_id UUID,
  old_values JSONB,
  new_values JSONB,
  ip_address VARCHAR(100),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

20. Tables minimales pour commencer le développement

20.1 Démarrage minimal

- roles
- users
- clients
- items
- invoices
- invoice_items
- payments
- commissions

20.2 Tables à ajouter ensuite

- bank_statements
- bank_transactions
- audit_logs
- permissions
- role_permissions

- settings

21. Résultat de l'étape 6

- les entités principales
- les tables de la base de données
- les relations entre les tables
- les clés primaires et étrangères
- les contraintes importantes
- les règles de calcul
- les règles d'archivage
- un schéma SQL simplifié
- les tables minimales pour démarrer le développement

22. Prochaine étape

La prochaine étape est l'Étape 7 - Architecture technique du système. Elle définira le frontend, le backend, la base de données, le stockage des fichiers PDF, le module OCR, l'authentification, la structure des dossiers du projet et les flux entre composants.